CS 0007: Introduction to Java Lecture 9

Nathan Ong
University of Pittsburgh
September 29, 2016

If Statements

- English usage?
- “If it will rain today, then I will bring my umbrella.”
- Rain today would indicate that I have brought my umbrella
- Cause and effect relationship: when a condition is met, a behavior is elicited.

Skeleton Statement

```
if(<boolean condition>)
{
    …
}
…
```

Boolean Condition

- Must evaluate to true in order for the statements within the `if`-block to execute.
- Evaluating to false will skip the block.

Java Example

```
  /*  checkWeather returns true if
  it
   *  will rain today.*/
boolean rainToday = checkWeather();
if(rainToday)
{
      bringUmbrella();
}
goToSchool();
```

if()

- if hasFacebook is true, then print "has a Facebook"

```
if(hasFacebook)
{
  System.out.println("has a
  Facebook");
}//end block if(hasFacebook == true)
```

- But how do I represent when hasFacebook is false?

else if()

- Otherwise, if hasFacebook is false, then print "does not have a Facebook"

```
else if(!hasFacebook)
{
  System.out.println("does not
      have a Facebook");
}//end block else if(hasFacebook
 // == false)
```

if()

```
if(hasFacebook)
{
  System.out.println("has a Facebook");
}//end block if(hasFacebook)
else if(!hasFacebook)
{
  System.out.println("does not have a
   Facebook");
}//end block else if(!hasFacebook)
```

Still kinda wordy...

`else if(!hasFacebook)`

- How many values can **`hasFacebook`** have?
- How many values can a **`boolean`** have?
- Why do we even need to check for ! **`hasFacebook`** when we know if it is not true, it should do that portion of code?
- ⇒ **`else`**

`else`

- When all else fails, execute everything inside this block.
- All boolean expressions from the previous `if` and `else if` blocks must all evaluate to false for this block to execute.
- In the **`hasFacebook`** example, because we know it can only hold two values, if it is not true, then it must be false.

else

```
if(hasFacebook)
{
  System.out.println("has a Facebook");
}//end block if(hasFacebook)
else
{
  System.out.println("does not have a
   Facebook");
}//end block else
```

STILL kinda wordy...

```
System.out.println("has a Facebook");
```

- Both lines use `"a Facebook"`
- How can we use this to our advantage?
- Why do we even want to do this?

Optimization

- Eventually, we get down all of that code into something very short
- Shorter code generally (in all the programming you will be doing) means a faster program and is easier to read
- `if` statements take time to evaluate
- If you had five million of them, your program would be painfully slow. Reduce the number of if statements, if you can.

Reducing Common Code

```
if(hasFacebook)
{
  System.out.print("has");
}//end block if(hasFacebook)
else
{
  System.out.print("does not have");
}//end block else
System.out.println(" a Facebook");
```

This is overkill for this example, but may prove useful in later situations. I can still make this even shorter.

?:

- Simple choice operator

(boolean expression) ? true return : false return;

- Only use this in very simple `if` cases

?:

```
(hasFacebook ? "has" : "does not have") + " a Facebook");
```

- Now we can get it all in one line!

Adding It in

```
public class Name
{
  public static void main(String[] args)
  {
  String firstName = "Nathan";
  char midInitial = 'R';
  String lastName = "Ong";
  int age = 19;
  boolean hasFacebook = true;
  System.out.println(firstName + " " + midInitial +    " " +
    lastName + ", age: " + age + ", " +     (hasFacebook ? "has" :
    "does not have") +
      " a Facebook");
  }//end method main
}//End class Name
```

What is the output?



Was it really worth it?
How messy was that code?

Trade-Offs

- Readability vs. Length
- Who even wants to read a four-line one-liner?
- Instead you should somehow tell the programmer what you mean (also so you don't forget)
- Remember the strange green thing at the end?

Comment Reminders

```
public class Name
{
  public static void main(String[] args)
  {
  String firstName = "Nathan";
  char midInitial = 'R';
  String lastName = "Ong";
  int age = 19;
  boolean hasFacebook = true;
  System.out.println(firstName + " " + midInitial +    " " +
    lastName + ", age: " + age + ", " +     (hasFacebook ? "has" :
    "does not have") +
       " a Facebook");
  }//end method main
}//End class Name
```

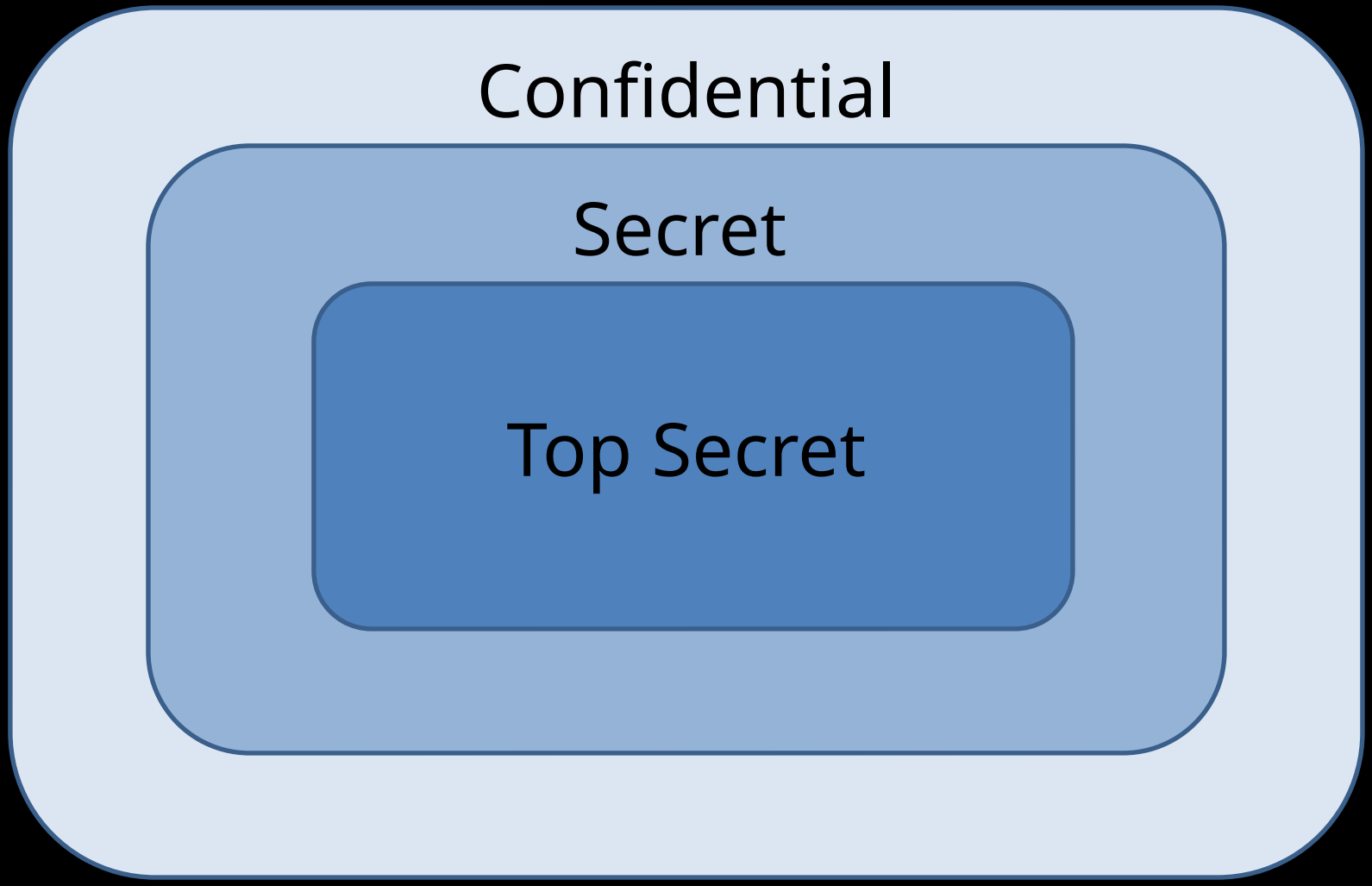
`if()` recap

```
//always run this code
if(condition1 is true)
{
	//run this code
}
else if(condition2 is true)
{
	//run this code
}
else if(condition3 is true)
{
	...
}
...
else //if conditions 1...n were not satisfied
{
	//run this code
}
//always run this code
```

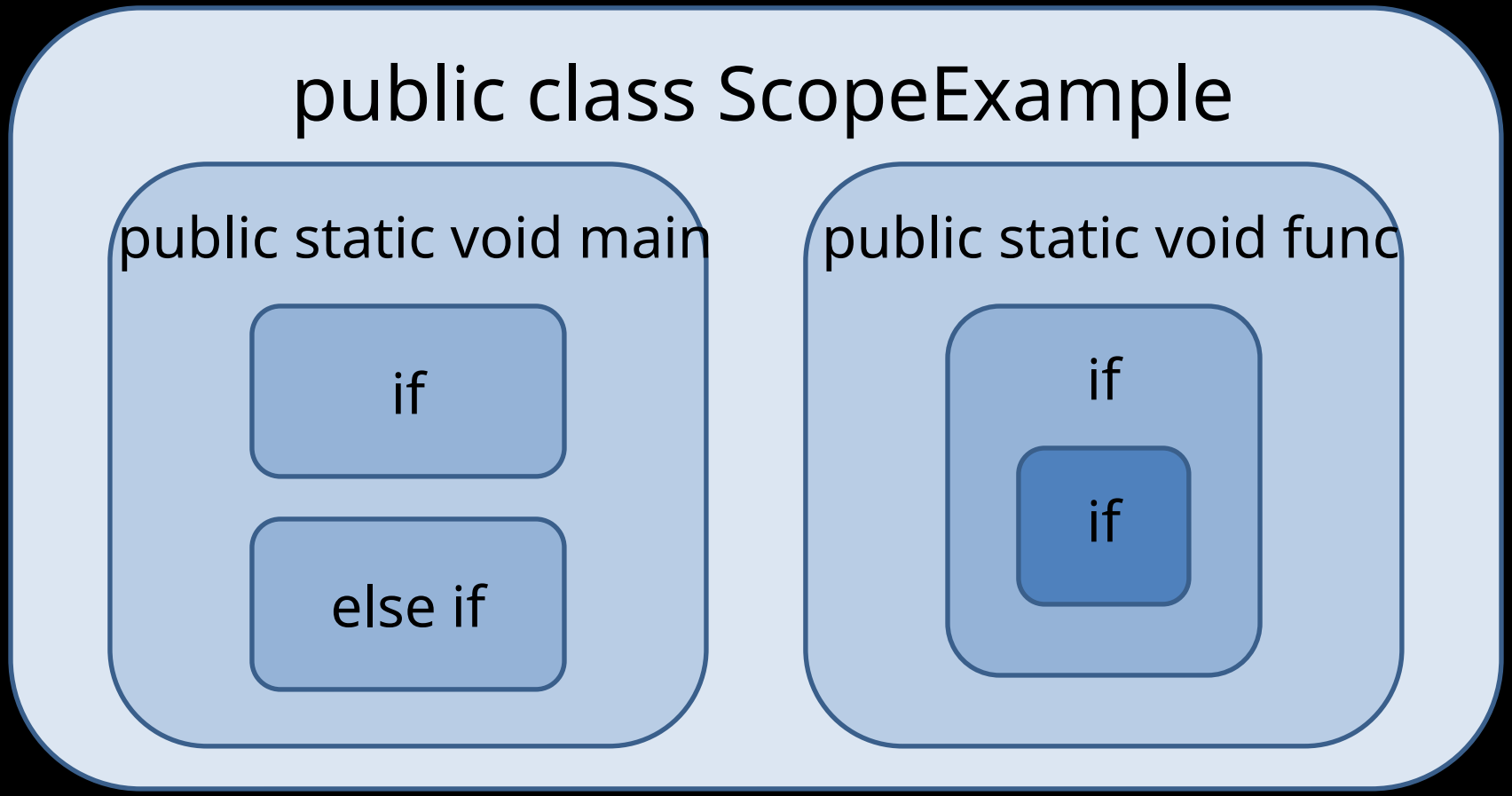
Revisiting Scope

- Scope works by curly brackets.
- Curly brackets surround all newly declared information, keeping it hidden from code outside of the curly brackets.

Scope



Scope



Darker shades can see lighter shades, but not vice versa

Scope in Code

```
public class Name
{
  public static void main(String[] args)
  {
  int x = 0;

  //x == 0, y is undefined
  if(x == 0)
  {
  int y = 10;
  x = 15;
  //x == 15, y == 10
  }
  //x == 15, y is undefined
  }//end method main
}//End class Name
```